

# HOSPITAL MANAGEMENT SYSTEM PROJECT

## INTRODUCTION

The **Hospital Management System (HMS)** is an integrated software platform designed to manage clinical, financial, and administrative operations. Modern healthcare facilities handle amounts of data including patient demographics, medical histories, appointment schedules, staff shifts, inventories, and billing transactions. This system replaces traditional paper-based record-keeping with an optimized, centralized relational database management system (RDBMS) to ensure data integrity, speed, security, and medical accuracy.

## PROJECT OBJECTIVES & SCOPE

- **Centralized Patient Tracking:** Maintain unique records for patients, monitoring their full journey from scheduled visits to historical medical evaluations.
- **Operational Transparency:** Track doctor specialties, departments, and staff allocation structures to maximize clinical resource utility.
- **Financial Integrity:** Guarantee distinct financial transactions matching one-to-one with concrete healthcare encounters to mitigate billing discrepancies.
- **Automated Pharmacy Records:** Log real-time medication inventories and trace precise pharmaceutical prescriptions back to specific consultations.

## SYSTEM BENEFITS

- **Data Consistency:** Relational constraints restrict invalid entries, ensuring orphan medical charts or billing statements cannot exist.
- **Speed & Accessibility:** Immediate access to patient background logs, test indices, and physician availabilities cuts down operational lag times.
- **Secure Scaling:** Modular entity structures make it seamless to append upcoming service workflows (e.g., telemetry logs or insurance pipelines) down the road.

## 2. SYSTEM ANALYSIS

### Existing System vs. Proposed System

- **Existing System:** Frequently relies on manual book entries or disconnected spreadsheet files. This structure results in redundant records, high human error margins during data duplication, difficulty retrieving patient backgrounds quickly, and vulnerable billing tracking.
- **Proposed System:** An automated, relational MySQL-backed architecture. It establishes centralized database schemas, forces input validation through specific keys, speeds up querying with index rules, and tracks complete medical lifetimes accurately.

### Feasibility Study

- **Technical Feasibility:** Developed using stable, open-source technology platforms (MySQL Database Server along with any standard frontend interface tool).
- **Operational Feasibility:** Clinical users require minimal training due to intuitive database abstractions matching real-world clinical department layouts.
- **Economic Feasibility:** Eliminates file storage space and administrative human overhead while safeguarding against unbilled treatments.

## 3. SYSTEM REQUIREMENT SPECIFICATION

## Hardware Requirements

- **Server Side:**
  - Processor: Minimum Intel Xeon or equivalent 4-core processor.
  - Memory: 8 GB RAM minimum.
  - Storage: 50 GB Solid State Drive (SSD) space for active transactional databases.
- **Client Workstation Side:**
  - Processor: Standard Intel Core i3 / AMD Ryzen 3 or higher.
  - Memory: 4 GB RAM.

## Software Requirements

- **Operating System:** Windows 10/11, Linux Ubuntu Server, or macOS.
  - **Database Management System:** MySQL Server 8.0 or newer.
  - **Database Management Tool:** MySQL Workbench, dBeaver, or phpMyAdmin.
  - **Development Stack (Optional Frontend/Backend Frameworks):** Java (Spring Boot), Python (Django/Flask), or Node.js paired with standard HTML5/CSS3/JavaScript components.
- 

## 4. SYSTEM DESIGN

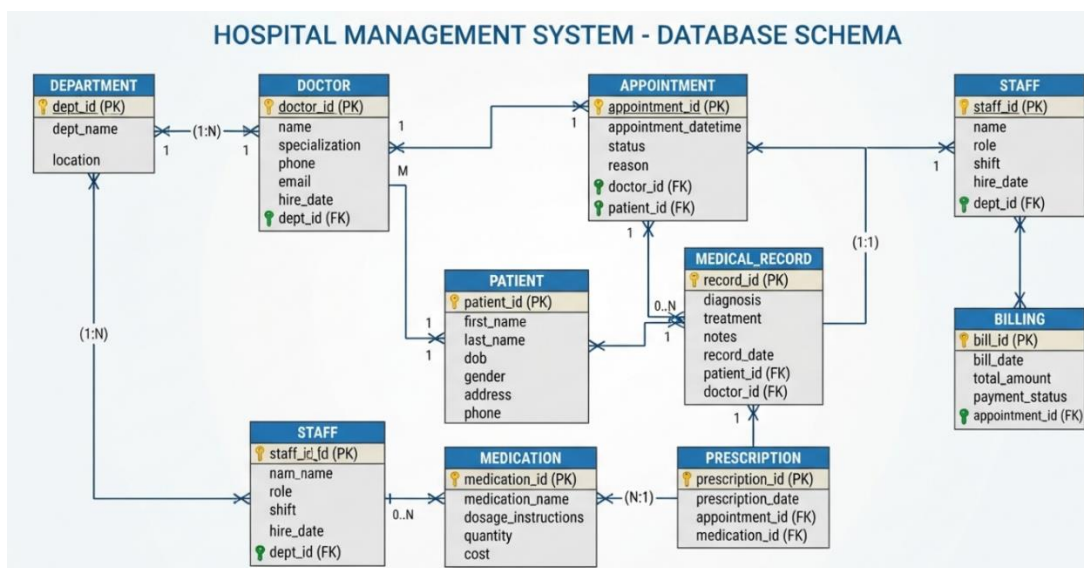
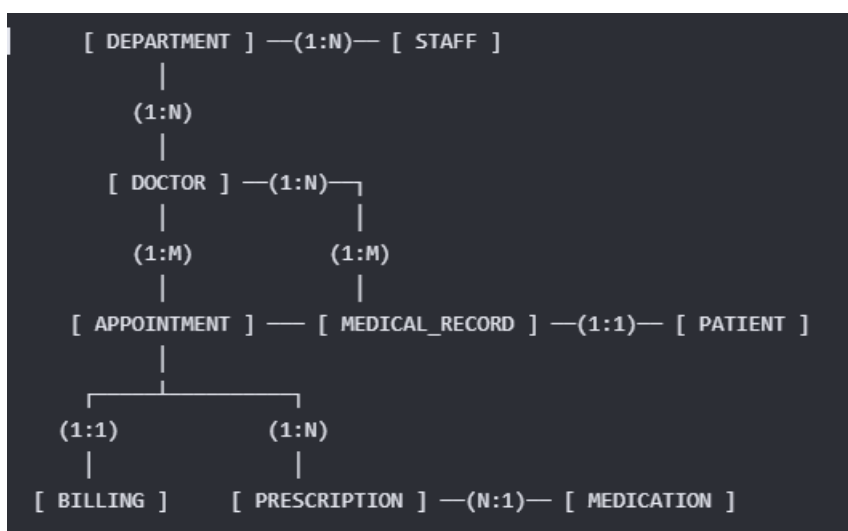
### Database Entity-Relationship Diagram (ERD) Structure

The database layout uses 9 key operational entities organized into specific operational domains:

- **Administration:** DEPARTMENT, STAFF, DOCTOR
- **Clinical Patient Flows:** PATIENT, APPOINTMENT, MEDICAL\_RECORD
- **Pharmacy & Accounting:** MEDICATION, PRESCRIPTION, BILLING

## Database Relations & Normalization

The system is normalized up to **Third Normal Form (3NF)** to ensure every data attribute relates explicitly to unique primary identification keys.



## 5. SYSTEM IMPLEMENTATION

### MySQL Table Creation Scripts (DDL)

## DEPARTMENT

```
CREATE TABLE DEPARTMENT (  
    dept_id INT AUTO_INCREMENT PRIMARY KEY,  
    dept_name VARCHAR(100) NOT NULL,  
    location VARCHAR(255)  
);
```

## PATIENT

```
CREATE TABLE PATIENT (  
    patient_id INT AUTO_INCREMENT PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    dob DATE,  
    gender VARCHAR(10),  
    address VARCHAR(255),  
    phone VARCHAR(20)  
);
```

## MEDICATION

```
CREATE TABLE MEDICATION (  
    medication_id INT AUTO_INCREMENT PRIMARY KEY,  
    medication_name VARCHAR(100) NOT NULL,  
    dosage_instructions TEXT,  
    quantity INT,  
    cost DECIMAL(10, 2)  
);
```

## STAFF

```
CREATE TABLE STAFF (  
    staff_id INT AUTO_INCREMENT PRIMARY KEY,
```

```
name VARCHAR(100) NOT NULL,  
role VARCHAR(50),  
shift VARCHAR(50),  
hire_date DATE,  
dept_id INT,  
FOREIGN KEY (dept_id) REFERENCES DEPARTMENT(dept_id) ON DELETE SET  
NULL  
);
```

## DOCTOR

```
CREATE TABLE DOCTOR (  
    doctor_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    specialization VARCHAR(100),  
    phone VARCHAR(20),  
    email VARCHAR(100),  
    hire_date DATE,  
    dept_id INT,  
    FOREIGN KEY (dept_id) REFERENCES DEPARTMENT(dept_id) ON DELETE SET  
    NULL  
);
```

## APPOINTMENT

```
CREATE TABLE APPOINTMENT (  
    appointment_id INT AUTO_INCREMENT PRIMARY KEY,  
    appointment_datetime DATETIME NOT NULL,  
    status VARCHAR(50),  
    reason TEXT,  
    doctor_id INT,  
    patient_id INT,  
    FOREIGN KEY (doctor_id) REFERENCES DOCTOR(doctor_id) ON DELETE CASCADE,  
    FOREIGN KEY (patient_id) REFERENCES PATIENT(patient_id) ON DELETE CASCADE
```

);

## MEDICAL\_RECORD

```
CREATE TABLE MEDICAL_RECORD (  
    record_id INT AUTO_INCREMENT PRIMARY KEY,  
    diagnosis TEXT,  
    treatment TEXT,  
    notes TEXT,  
    record_date DATE,  
    patient_id INT,  
    doctor_id INT,  
    FOREIGN KEY (patient_id) REFERENCES PATIENT(patient_id) ON DELETE CASCADE,  
    FOREIGN KEY (doctor_id) REFERENCES DOCTOR(doctor_id) ON DELETE CASCADE  
);
```

## BILLING

```
CREATE TABLE BILLING (  
    bill_id INT AUTO_INCREMENT PRIMARY KEY,  
    bill_date DATE NOT NULL,  
    total_amount DECIMAL(10, 2) NOT NULL,  
    payment_status VARCHAR(50),  
    appointment_id INT,  
    FOREIGN KEY (appointment_id) REFERENCES APPOINTMENT(appointment_id) ON  
DELETE CASCADE  
);
```

## PRESCRIPTION

```
CREATE TABLE PRESCRIPTION (  
    prescription_id INT AUTO_INCREMENT PRIMARY KEY,  
    prescription_date DATE NOT NULL,  
    appointment_id INT,  
    medication_id INT,
```

```
    FOREIGN KEY (appointment_id) REFERENCES APPOINTMENT(appointment_id) ON  
    DELETE CASCADE,  
    FOREIGN KEY (medication_id) REFERENCES MEDICATION(medication_id) ON  
    DELETE CASCADE  
);
```

## **Conclusion & Summary**

You have successfully mapped out, constructed, and queried a complete **Hospital Management System Database**.

**\*\* What Was Accomplished:\*\***

1. **Schema Design:** Verified and built a 9-table relational database architecture handling departments, staff, doctors, patients, appointments, medical records, billing, medications, and prescriptions.
2. **Data Integrity:** Populated the tables with **25 complex mock records** spread across **5 core departments**, strictly honoring relational dependency constraints (PK → FK).
3. **Query Engineering:** Developed solutions for **30 core operational requirements**, ranging from simple record selection to advanced data manipulation, multi-table structural JOIN operations, window analysis functions, and financial tracking logic.